

Wingman

A Deterministic Safety Supervisor for GPU-Based Perception, Built on the PHYTEC
phyBOARD-RT1170

Submission — All About Circuits / PHYTEC / NXP 2026 Embedded Design Contest

Suvir — Department of Computer Engineering, University of Alberta

*A Jetson GPU perceives hazards; a PHYTEC phyBOARD-RT1170 makes the safety decision —
deterministically — and fails safe the moment perception goes silent.*

Abstract

Wingman demonstrates a general pattern for safety-critical robotics: separating perception from safety decision-making across two processors, so that a fast but non-deterministic GPU vision system can never itself become a single point of failure. This split is the same one underlying production vehicle safety systems, industrial collaborative-robot (cobot) safety controllers, and autonomous mobile robot (AMR) collision-avoidance systems. Wingman implements it as an automotive-styled advanced driver-assistance (ADAS) *prototype* — a proof of concept, not a certified system, chosen because collision timing requirements are concrete and easy to demonstrate on a bench. An NVIDIA Jetson TX2 performs GPU-accelerated object detection and tracking; a PHYTEC phyBOARD-RT1170 (NXP i.MX RT1176, Zephyr RTOS) independently evaluates collision risk and enforces a fail-safe response, entirely on deterministic, watchdog-protected hardware. This document reports the system design and results measured on physical hardware, including a full fault-injection test matrix, decision-latency and failover timing, a CUDA kernel benchmark, and a real firmware bug found and corrected in the board's own PHYTEC BSP branch.

Keywords: advanced driver-assistance systems, functional safety, real-time embedded systems, Zephyr RTOS, sensor fusion, fail-safe design, heterogeneous computing.

1. The Problem

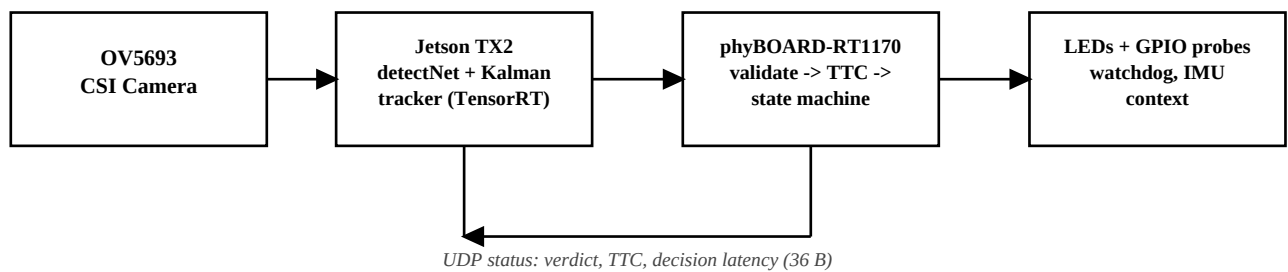
GPU-based perception is fast and capable, but it is not deterministic: it can stall on an unexpected frame, crash, drop the camera feed, or simply take longer than expected under load. A safety system built entirely on that foundation inherits its uncertainty — a problem that is identical whether the perception system is watching for pedestrians from a vehicle, for people near a robot arm on a factory floor, or for obstacles ahead of an autonomous mobile robot in a warehouse. Each of these domains solves it the same way: a perception computer determines *what* is present in the environment, while a simpler, independently-watchdogged control unit determines *whether it is dangerous* and what action to take if the perception system becomes unavailable. This is the split required by automotive safety practice and by industrial functional-safety standards such as IEC 61508 and ISO 13849 alike. Wingman implements it at prototype scale, in an automotive-styled demonstration.

2. System Architecture

Table 1. Division of responsibility between the two processors.

	Perception — Jetson TX2	Supervisor — phyBOARD-RT1170
Platform	Linux, CUDA / TensorRT	Zephyr RTOS, 996 MHz Cortex-M7
Role	Heavy, parallel computer vision	Bounded-time decision-making
Failure mode	May stall, crash, or drop frames	Monitored by a hardware watchdog
Responsibility	Determines what is present	Determines whether it is dangerous, and the response if perception is lost

*UDP perception frame
146 B @ 30 Hz (heartbeat)*



The TX2 overlays detection boxes in the colour of the RT1170's verdict.

Figure 1. Data flow between the perception and supervisor subsystems.

3. Why the phyBOARD-RT1170

Wingman deliberately positions the phyBOARD-RT1170 as the deterministic authority to which the GPU-based perception system reports, rather than as a passive peripheral. Every safety decision, and the fail-safe response when perception is lost, executes entirely on the RT1170's Cortex-M7, independent of the Linux/GPU subsystem's state. The properties measured below — cycle-accurate timing, a hardware watchdog, and bounded fail-safe response — are exactly the properties any perception-plus-supervisor safety architecture needs, whether the perception input comes from a vehicle camera, a warehouse robot's LIDAR, or a drone's obstacle sensor.

Deterministic, cycle-accurate timing. Decision latency is measured using the M7's own 996 MHz cycle counter rather than wall-clock estimation: mean 49.79 μ s, 99th-percentile 118.62 μ s, maximum 205.10 μ s across 1,614 measured frames.

Hardware watchdog (WDOG1). Fed exclusively by the highest-priority decision thread and demonstrated live via a deliberate thread stall: the board resets within approximately 0.5 seconds and reports the reset cause — read from the SoC's SRC register — on the subsequent boot.

Real-time Ethernet connectivity. The board's 1 Gbit ENET_1G interface (verified full-duplex, carrier active) carries a custom low-latency UDP protocol; round-trip latency to the TX2 averages 0.87 ms with a maximum of 2.02 ms across 1,614 frames.

Fail-safe behavior by design. Any interruption of valid perception data — network congestion, a crashed camera pipeline, or corrupted frames — triggers a FAILSAFE state within a single heartbeat window. Across 50 independent trials: mean 107.26 ms, maximum 112.36 ms, consistent with the design target of approximately 100 ms plus one 10 ms heartbeat interval.

Onboard IMU (ICM-40627) integration. A defect was identified and corrected in this board's PHYTEC BSP branch (v4.1.0-phy2): the IMU devicetree entry specifies an incorrect I2C address. PHYTEC's own upstream repository contains a corresponding fix on a newer branch; the same one-line address correction was applied here via a targeted devicetree overlay rather than a full BSP upgrade. Sensor axis behavior was subsequently verified against physical motion of the board.

Complete fault-injection test coverage. All fourteen test scenarios — six scripted scenarios and every fault-injection variant, including packet loss, CRC corruption, reordering, duplication, rate variation, and simulated perception failure — passed on physical hardware, matching the result obtained under native simulation.

The Jetson TX2 supplies perception input and is, by design, a secondary component in this evaluation: every safety-critical measurement reported here was obtained on the phyBOARD-RT1170 alone.

4. System Design

4.1 Communication Protocol

Perception frames are 146 bytes and encode up to eight tracked objects (identifier, class, confidence, distance, lateral offset, closing speed). Status replies are 36 bytes. Both use little-endian encoding, CRC-16/CCITT-FALSE integrity checking, and monotonic sequence numbers. Frames are transmitted on every camera cycle regardless of detection count, serving as the supervisor's heartbeat signal.

4.2 Supervisor Threading Model

The RT1170 firmware runs four cooperating threads. *net_rx* timestamps each incoming datagram at the moment of socket delivery, validates length, CRC, magic number, and protocol version, and enqueues valid frames. *decide*, the highest-priority thread, executes the state machine, drives status LEDs, measures decision latency, and services the hardware watchdog. A 10 ms timer thread detects heartbeat loss. *status_tx* transmits the reply packet to the originating host.

4.3 Decision Logic

Tracks are considered in-path when lateral offset is under 1.5 m, confidence is at least 0.4, and closing speed exceeds 0.1 m/s. Time-to-collision (TTC) is computed as distance divided by closing speed; the track with minimum TTC is designated the primary threat. State transitions use hysteresis: CAUTION below 2.5 s (exit above 2.8 s), WARNING below 1.2 s (exit above 1.5 s), with an additional close-range override for objects within 3 m and closing, independent of computed TTC. Any state transitions to FAILSAFE after 100 ms without a valid frame or five consecutive corrupt frames; recovery to normal operation requires ten consecutive

valid, in-sequence frames.

4.4 Inertial Measurement Context

Sustained longitudinal deceleration below -0.3 g for 200 ms downgrades a WARNING state to CAUTION, reflecting driver-initiated braking, except during the close-range override condition. A lateral swerve during an active threat is logged as a near-miss event; an impact exceeding 2 g latches a 64-decision black-box buffer.

4.5 Perception Pipeline

Object detection uses SSD-MobileNet-v2 via TensorRT. Monocular distance is estimated by the pinhole relation $\text{distance} = f \cdot H / h$, where f is the calibrated focal length in pixels, H is an assumed real-world object height by class, and h is the detected bounding-box height in pixels. A constant-velocity Kalman filter with gated nearest-neighbor data association performs tracking, requiring three consecutive detections before a track is reported.



Figure 2. A frame captured directly by the TX2's OV5693 CSI camera during initial hardware verification, prior to any perception software being run, confirming the camera pipeline was functioning correctly.

5. Results

All figures below were obtained on physical hardware — the phyBOARD-RT1170 and Jetson TX2 — rather than simulation. Complete raw data accompanies the project source.

Table 2. Measured results summary, all obtained on physical hardware.

Metric	Result
Decision latency (socket delivery to output)	n = 1,614; min 11.92, mean 49.79, p99 118.62, max 205.10 μ s
Failover time (last valid frame to FAILSAFE)	n = 50, 50/50 passed; min 102.32, mean 107.26, p99 112.36, max 112.36 ms
Round-trip latency (TX2 to RT1170 to TX2)	n = 1,614; min 0.50, mean 0.87, p99 1.37, max 2.02 ms

Scenario / fault-injection test matrix	14 of 14 passed — six scenarios and every fault variant; matches native-simulation result
Latency under network flood (5,000 packets/s)	RT1170 decision latency unaffected: 51.19 vs. 47.46 μ s baseline mean. A host-side packet-delivery irregularity was observed during the flood interval (21/30 vs. 20/20 scenario passes), attributed to CPU/socket contention on the TX2 host rather than the RT1170 firmware.
Perception frame rate	Approximately 30.2 fps sustained (441 samples) over a 10-minute continuous live-camera evaluation, with no failures
CUDA Kalman-filter kernel (Jetson TX2, sm_62)	CPU outperforms GPU at $N = 64$ (ratio 0.91); GPU becomes favorable at $N = 256$ (3.33 $\times$); scales to 59–61 $\times$ at $N \geq 16,384$. Structure-of-arrays layout achieves 100% global memory efficiency versus 14.04% / 12.50% for array-of-structures (measured via nvprof), at comparable occupancy



Figure 3. The substitute calibration procedure described in Section 6: a sheet of standard letter paper (0.2794 m, portrait orientation) held at two measured distances from the camera, 1.0 m (left) and 1.8 m (right). Focal length was computed from the paper's measured pixel height at each distance; the two independent estimates differed by 13.3%, which is reported as a limitation rather than resolved by additional averaging. The subject's face is intentionally obscured by the paper in both frames.

6. Limitations

- Monocular distance estimation assumes a fixed real-world height per object class; unusually-sized or partially-occluded objects will produce inaccurate distance estimates.
- The available test environment could not accommodate a fully-framed standing person at any usable camera distance, so the standard three-distance person-based calibration procedure could not be executed. A substitute procedure using a known-dimension object (a sheet of standard letter paper) at two measured distances was used instead, yielding a 13.3% spread between the two focal-length estimates — above the calibration tool's own 10% warning threshold, most plausibly attributable to imprecision in holding the reference object level by hand. Reported distance and time-to-collision values should be regarded as approximate.
- Object tracking exhibits reduced stability at very close range (approximately arm's length), where the detector intermittently loses lock. This can cause the state machine to revert to NOMINAL abruptly, reflecting loss of a confirmed detection rather than resolution of the underlying hazard.

- The IMU's sign convention — which physical direction corresponds to braking versus acceleration — depends on final vehicle-mounting orientation and was intentionally left unconfigured pending that decision. Axis *assignment* (distinguishing pitch from roll) was independently verified by physically manipulating the board and observing live sensor output.
- Reported latency figures are self-reported by the RT1170's internal cycle counter; no external logic analyzer was available to independently corroborate these measurements against the two GPIO timing probes provided for that purpose.
- This is a bench-evaluated prototype using live and replayed video. It is not qualified to ISO 26262 and is not a commercial product.

7. Future Work

7.1 Applications Beyond Automotive

Because the RT1170 side of Wingman makes no automotive-specific assumptions — it consumes tracked objects, distance, and closing speed over a generic UDP link, and outputs a hazard-level verdict — the identical supervisor firmware applies directly to other safety-critical robotics domains with only the perception front-end and the physical output changed. Concrete candidates: **industrial cobot safety**, where a vision system flags human proximity to a robot arm and the RT1170 enforces the stop/slow decision independently of the robot's main controller, mirroring the perception/safety split already required by IEC 61508 and ISO 13849; **autonomous mobile robot (AMR) collision avoidance** in warehouse settings, using the same TTC-and-close-range-override logic against forklifts and pedestrians; and **small UAV obstacle avoidance**, where the same watchdog-backed fail-safe pattern already used in flight controllers could be driven by a GPU-based vision front end instead of the more limited sensors typical of that role today. None of these have been built or tested here; they follow directly from the architecture as designed.

7.2 Automotive-Specific Next Steps

Repeat camera calibration using the standard person-based procedure in a larger test space to obtain verified distance and TTC accuracy.

Perform an independent logic-analyzer cross-check of decision-latency and failover measurements.

Finalize IMU mounting orientation and configure the corresponding sign convention for a specific vehicle installation.

Implement a TinyML driving-maneuver classifier operating on IMU windows (steady, braking, accelerating, swerving); scaffolded but not completed within the contest timeline.

Employ the RT1170's secondary Cortex-M4 core as an independent hardware monitor of the M7's heartbeat, via the Messaging Unit / inter-processor mailbox.

Investigate radar or GPS sensor fusion, and a custom carrier board sized to the 30 × 30 mm phyCORE-RT1170 module.

8. Bill of Materials

Table 3. Hardware and software used to build and test the prototype.

Category	Items
Hardware	PHYTEC phyBOARD-RT1170 development kit with 5 V USB-C power supply; NVIDIA Jetson TX2 developer kit with 19 V power supply; Cat5e/6 Ethernet cable; two micro-USB cables (console and USB-OTG programming); OV5693 CSI camera module (included with TX2 kit)
Software and Toolchain	Zephyr RTOS 4.1 (PHYTEC BSP v4.1.0-phy2); Zephyr SDK 0.17.0; NXP SPSDK / blhost (USB-OTG programming, no debug probe required); NVIDIA JetPack 4.6.6 (L4T R32.7.6, CUDA 10.2, TensorRT 8.2); jetson-inference; Python 3

9. Source Code and Data Availability

Complete source code, build and programming instructions, a step-by-step hardware bring-up procedure, and all raw measurement data referenced in this document are available at: github.com/SiliconBeast/wingman.

10. References

- 1 PHYTEC Messtechnik GmbH, *BSP Zephyr RT1170 (PD25.1.1) Documentation*, docs.phytec.com/projects/zephyr-phycore-rt1170.
- 2 Zephyr Project, *phyBOARD-Atlas i.MX RT1170 Board Documentation*, docs.zephyrproject.org/latest/boards/phytec/phyboard_atlas.
- 3 NXP Semiconductors, *i.MX RT1170 Crossover MCU Reference Manual*, document IMXRT1170RM.
- 4 NXP Semiconductors, *MCUXpresso SPSDK / blhost User's Guide* (USB-OTG serial bootloader programming).
- 5 International Electrotechnical Commission, *IEC 61508: Functional Safety of Electrical/Electronic/Programmable Electronic Safety-Related Systems*.
- 6 International Organization for Standardization, *ISO 13849: Safety of Machinery — Safety-Related Parts of Control Systems*.
- 7 Suvir, *Wingman* (source code, build instructions, and raw measurement data), github.com/SiliconBeast/wingman, 2026.